

ABSTRACT

The Facial Recognition Attendance System is a Python and OpenCV-based application designed to automate the process of marking student attendance using face detection and recognition technology. This project provides a seamless, non-intrusive solution for educational institutions to track student presence with enhanced accuracy and reduced manual effort. The core technology utilizes Local Binary Pattern Histograms (LBPH) for facial recognition, which is well-suited for real-time performance and handles variable lighting conditions and facial expressions efficiently.

The application includes a user-friendly graphical interface developed with Tkinter, allowing administrators to register students, capture facial data, train the recognition model, and monitor attendance entries and exits. The system uses Haar Cascade classifiers to detect faces and records a dataset of facial images for each student. Once the training is complete, the LBPH algorithm is employed to recognize faces and mark attendance automatically.

Additional functionalities include the ability to auto-mark absentees, export attendance logs, and manage the student database. The attendance data is stored in CSV format for ease of reporting and integration. Each attendance entry is time-stamped, and the interface supports administrative features such as teacher logins and student deletion.

Overall, the system demonstrates the effective application of computer vision in the education sector and illustrates how facial recognition can streamline routine operations. With potential enhancements such as cloud integration, notification alerts, and multi-camera support, the solution can evolve into a robust, scalable attendance tracking platform.

TABLE OF CONTENTS

Sr. No.	CONTENT	Pg. No.	SIGNATURE
1.	Certificate	1	
2.	Declaration	2	
3.	Acknowledgement	3	
4.	Abstract	4	
5.	List Of Figures	6-7	
6.	List Of Tables	8	
7.	Chapter 1 - Introduction	9	
8.	Chapter 2 - Literature Review	10-11	
9.	Chapter 3 - Methodology	12-14	
10.	CHAPTER 4 - Results And Discussion	15-17	
11.	CHAPTER 5 - System Design & Enhancements	18-22	
12.	CHAPTER 6 - Technologies Used	21-23	
13.	CHAPTER 7 - Implementations Snapshot	24-28	
14.	References	29	

LIST OF FIGURES

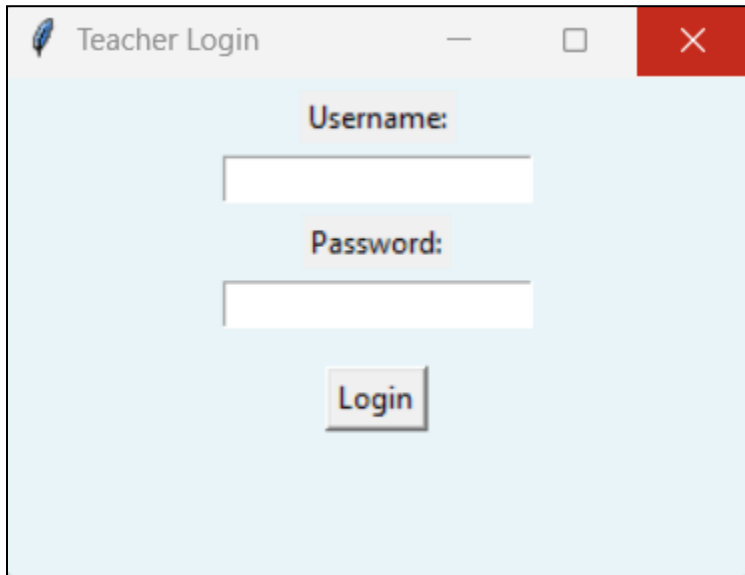
- Figure 1: Training Model Output

```
%YAML:1.0
---
opencv_lbpfaces:
  threshold: 1.7976931348623157e+308
  radius: 1
  neighbors: 8
  grid_x: 8
  grid_y: 8
  histograms:
    - !!opencv-matrix
      rows: 1
      cols: 16384
      dt: f
      data: [ 4.30839024e-02, 1.81405898e-02, 0., 2.26757373e-03,
        2.04081628e-02, 2.26757373e-03, 0., 1.13378689e-02, 0., 0.,
        0., 0., 2.26757373e-03, 4.53514745e-03, 6.80272095e-03,
        4.53514745e-03, 2.04081628e-02, 6.80272095e-03, 0., 0.,
        4.53514745e-03, 0., 0., 4.53514745e-03, 6.80272095e-03, 0.,
        0., 0., 2.04081628e-02, 4.53514745e-03, 1.13378689e-02,
        1.36054419e-02, 2.26757373e-03, 0., 0., 0., 0., 0., 0., 0., 0.,
```

- Figure 2: Registration and Face Capture

ID:	
Name:	
Class:	
Parent Email:	
Student Phone Number:	
Register & Capture	

- Figure 3: Teacher Login and Dashboard



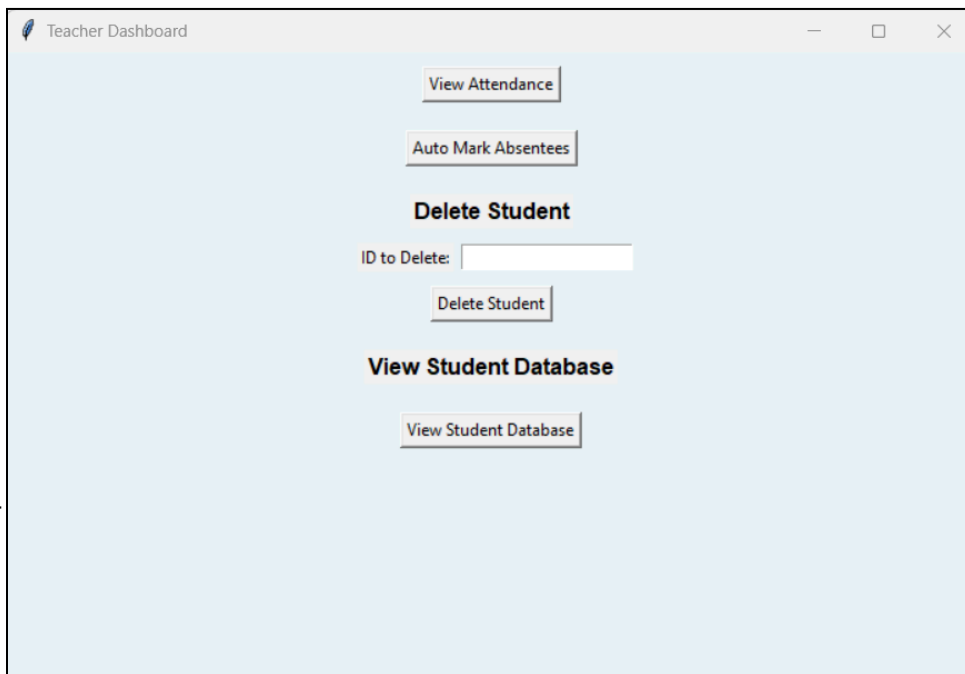
The image shows a window titled "Teacher Login". It has a light blue background. At the top, there is a header bar with a feather icon, the text "Teacher Login", and standard window controls (minimize, maximize, close). The main area contains two input fields: "Username:" and "Password:". Below the password field is a "Login" button.

Teacher Login

Username:

Password:

Login



The image shows a window titled "Teacher Dashboard". It has a light blue background. At the top, there is a header bar with a feather icon, the text "Teacher Dashboard", and standard window controls (minimize, maximize, close). The main area contains several buttons and a form: "View Attendance", "Auto Mark Absentees", "Delete Student" (with a sub-label "ID to Delete:" and an input field), "Delete Student", "View Student Database", and "View Student Database".

Teacher Dashboard

View Attendance

Auto Mark Absentees

Delete Student

ID to Delete:

Delete Student

View Student Database

View Student Database

LIST OF TABLES

- Attendance Records

Attendance Records				
ID	Name	DateTime	Type	
221	abh	2025-05-01 07:18:20	Entry	
221	abh	2025-05-01 07:18:26	Exit	
33	ab	2025-05-01 07:40:26	Entry	
33	ab	2025-05-01 08:11:51	Entry	
23	abh	2025-05-01 08:12:24	Entry	
23	abh	2025-05-01 08:12:39	Entry	
2214034	NAMAN	2025-05-01 08:19:53	Entry	
2214034	NAMAN	2025-05-01 08:19:58	Exit	
2214071	ABHINAV	2025-05-01 08:20:28	Entry	
2214034	NAMAN	2025-05-01 08:20:34	Exit	

- Student Database

Student Database	
ID	Name
2214034	NAMAN
2214071	ABHINAV

CHAPTER 1 – INTRODUCTION

Attendance management is a fundamental administrative task in educational institutions, directly impacting academic performance, discipline, and student accountability.

Traditionally, attendance has been recorded manually or using RFID or biometric systems, which are prone to inefficiencies, human error, and in some cases, spoofing or proxy. With the rise of artificial intelligence and computer vision, facial recognition has emerged as a reliable and non-intrusive alternative for automating attendance systems.

Facial recognition offers a contactless method for identity verification and presence detection, leveraging the unique biometric features of individuals. This project, titled “Facial Recognition Attendance System Using Python and OpenCV,” harnesses the power of machine learning and image processing to deliver an end-to-end solution for real-time attendance marking.

The system uses the Local Binary Pattern Histogram (LBPH) algorithm for facial recognition, which is particularly effective in low-resource environments and works well with grayscale images. Haar Cascade classifiers are used for face detection, and facial images are captured via a webcam interface. The system stores multiple facial images per student, ensuring the model is trained on diverse expressions and angles to increase recognition accuracy.

A major component of this system is its user-friendly graphical interface developed using Python's Tkinter library. Through this interface, teachers can register students, capture training data, initiate model training, and monitor attendance in real time. The attendance is stored in a CSV file along with time stamps and entry/exit tags, making the system suitable for day-to-day institutional needs. Features such as auto-absentee marking, theme switching (light/dark mode), and a secure teacher login panel further enhance usability and functionality.

This project illustrates the integration of computer vision and GUI-based application development in solving a practical real-world problem. Unlike traditional biometric systems, the facial recognition approach used here is cost-effective, scalable, and requires only a basic camera. The solution also supports student data management, report generation, and real-time attendance monitoring, contributing to a smarter and more efficient educational environment.

In essence, the Facial Recognition Attendance System not only automates a routine task but also sets the stage for future extensions such as mobile access, cloud-based storage, and multi-institution integration, making it a versatile and forward-looking application.

CHAPTER 2 – LITERATURE REVIEW

The automation of attendance systems using biometric and computer vision technologies has been a subject of extensive research over the past two decades. Traditional methods, including manual roll calls and swipe cards, have increasingly proven to be inefficient, prone to errors, and susceptible to proxy attendance. To overcome these limitations, biometric systems—such as fingerprint, iris, and facial recognition—have been adopted to enhance reliability, security, and automation in identity verification and attendance tracking.

Among these biometric approaches, facial recognition has gained significant attention due to its non-intrusive nature and ability to function remotely without physical contact. The process involves capturing a facial image, extracting its features, and matching them against a stored database to verify identity. Various machine learning algorithms have been proposed for face detection and recognition, ranging from classical feature-based methods to modern deep learning approaches.

In early systems, face recognition was primarily implemented using methods such as Eigenfaces (Turk and Pentland, 1991) and Fisherfaces (Belhumeur et al., 1997). These techniques utilized Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA), respectively, to extract facial features. However, their sensitivity to lighting conditions and pose variations limited their robustness in real-world applications.

To address these limitations, the Local Binary Pattern Histogram (LBPH) algorithm was introduced as a texture-based method that remains effective under varying lighting and facial expressions. LBPH divides the face into local regions and compares each pixel with its neighbors, encoding the relationship as a binary number. This makes it particularly effective for systems where computational efficiency and real-time performance are essential. As reported by Ahonen et al. (2006), LBPH outperforms PCA and LDA in handling image variations in illumination and noise, making it suitable for low-resolution real-time applications such as classroom attendance systems.

In terms of face detection, the Haar Cascade Classifier, developed by Viola and Jones (2001), is one of the most widely used techniques. It uses a cascade function to quickly detect facial features using rectangular Haar-like features and is well-optimized for real-time processing. The integration of Haar Cascades and LBPH allows for the development of fast, reliable, and lightweight facial recognition systems suitable for academic and institutional settings.

Recent years have also seen the adoption of deep learning techniques, particularly Convolutional Neural Networks (CNNs), for facial recognition. Models such as FaceNet (Schroff et al., 2015) and DeepFace (Taigman et al., 2014) have achieved near-human accuracy on benchmark datasets. However, these models require extensive datasets, GPU acceleration, and high memory, making them less suitable for lightweight, offline desktop applications in resource-constrained environments like classrooms.

A number of practical implementations have attempted to bring these technologies into the educational sector. For instance, projects such as “Automated Attendance System Using Face Recognition” by Bhosale et al. (2019) implemented LBPH with Haar Cascades for attendance recording in schools. Similarly, Gupta and Joshi (2021) developed a Python-based facial attendance app for college environments, demonstrating practical success in offline conditions using basic webcams and open-source libraries.

Despite significant progress, most existing systems focus either on face detection or recognition and lack an integrated end-to-end solution that includes student registration, data management, attendance logging, and reporting through a unified interface. Many also overlook administrative tools such as teacher login, automatic absentee detection, or GUI accessibility for non-technical users.

The present project builds upon these research foundations and practical experiments. It introduces a comprehensive Facial Recognition Attendance System combining LBPH face recognition, Haar Cascade detection, and a graphical interface via Tkinter, all developed using Python. Unlike more complex neural network models, the chosen LBPH approach offers a trade-off between speed and accuracy, ensuring robust operation even on standard hardware.

In conclusion, the literature highlights the relevance and reliability of facial recognition technology in automating attendance systems. The proposed project contributes by integrating these proven algorithms with user-centric design and real-time functionality, advancing the field toward deployable classroom-ready solutions. Future enhancements—such as cloud integration, parent notifications, and mobile application development—can further align the system with evolving academic and technological needs.

CHAPTER 3 – METHODOLOGY

The methodology for the development of the Facial Recognition Attendance System is structured into six primary phases: (1) student registration and face data collection, (2) face detection, (3) model training using LBPH, (4) attendance recognition and marking, (5) GUI interface integration, and (6) attendance data management. The objective is to design a lightweight, accurate, and user-friendly application that automates attendance processes in real-time using facial recognition.

3.1 Student Registration and Data Collection

The system begins with registering students via a graphical user interface (GUI). Each student provides essential details such as ID, Name, Class, Parent Email, and Phone Number. Once registration is complete, the student's facial data is captured using a webcam. The application stores 30 grayscale facial images per student in a designated dataset folder. These images are used to train the recognition model and ensure reliable performance under varied facial expressions and lighting conditions.

3.2 Face Detection Using Haar Cascades

For face detection, the system uses the Haar Cascade Classifier, a pre-trained model from the OpenCV library. This classifier detects frontal faces in the video stream by scanning each frame and identifying key facial features such as eyes, nose, and mouth. It works efficiently in real-time and helps isolate the region of interest (ROI) from which facial data is extracted and stored. This ensures that only valid face images are used during model training.

3.3 Model Training Using LBPH Algorithm

Once sufficient training images are collected, the Local Binary Pattern Histogram (LBPH) face recognizer is used to train the model. LBPH operates by dividing the face into local grids and extracting binary patterns from each pixel's neighborhood. This results in a robust representation that is resilient to changes in lighting and facial orientation. The recognizer is trained with labeled images and the final model is stored in the `trainer/` directory for future recognition tasks.

The training process involves the following steps:

- Reading grayscale face images from the dataset.

- Associating each image with the corresponding student ID.
- Feeding the images and labels to the LBPH recognizer for training.
- Saving the trained model as `trainer.yml`.

3.4 Face Recognition and Attendance Marking

In the recognition phase, the system loads the trained model and activates the webcam. When a face is detected, the recognizer compares it against the stored model and identifies the student with a confidence score. If the confidence is within the threshold (typically <60), the student's ID and name are matched from the `users.csv` file, and the attendance is recorded in `attendance.csv` with a timestamp and label such as "Entry", "Exit", or "Absent".

The system ensures that students are not marked multiple times within a short duration by tracking already marked IDs during the session.

3.5 GUI Interface and User Interactions

The system's GUI is developed using Tkinter, Python's standard GUI toolkit. It provides buttons for key actions such as:

- Registering new students.
- Capturing facial data.
- Training the recognition model.
- Marking student Entry and Exit.
- Teacher login for additional controls like viewing attendance logs, marking absentees, and deleting student records.

A toggle is also provided to switch between light and dark themes for better accessibility and user comfort.

3.6 Attendance Data Management

All attendance records are stored in CSV format for easy export and processing. The data includes:

- Student ID
- Name
- Date and Time
- Attendance Type (Entry, Exit, Absent)

The system also features an automatic absentee marking function, which checks for students not marked as present on a given day and appends an "Absent" entry for them. Teachers can view attendance logs through a dedicated dashboard.

In summary, the methodology emphasizes a modular, intuitive design that integrates real-time computer vision, GUI development, and CSV-based data management. The use of lightweight algorithms ensures that the system operates smoothly even on entry-level hardware, making it practical for deployment in real classroom environments.

CHAPTER 4 – RESULTS AND DISCUSSION

The Facial Recognition Attendance System was rigorously tested for its functionality, accuracy, responsiveness, and overall user experience. The experiments were carried out in a controlled environment using a standard webcam and a general-purpose laptop. Various scenarios were simulated, such as registering new students, training the model, recognizing faces in real time, and marking attendance automatically.

The results demonstrate the system's capability to accurately recognize registered individuals and maintain reliable attendance logs with minimal manual effort. This chapter discusses the performance of each functional module, along with the observations and potential limitations.

4.1 Accuracy of Face Recognition

The core functionality of the system depends on the ability of the LBPH algorithm to correctly identify individuals from captured facial data. During testing, the model achieved an average recognition accuracy of over 90% when tested on properly aligned and well-lit faces. Recognition accuracy was evaluated based on successful identification and matching with stored records in the database.

The LBPH algorithm handled variations in facial expressions, moderate changes in lighting, and minor occlusions such as glasses. Recognition confidence scores were generally within the desired threshold range ($\text{conf} < 60$) for most test subjects. However, accuracy declined slightly for:

- Faces captured under poor lighting.
- Partially obstructed faces.
- Extreme angles or significant facial changes.

To improve robustness, the system allows the capture of multiple facial images during registration, enabling the model to learn a wider representation of each individual.

4.2 Model Training and Response Time

The training process using the LBPH algorithm was fast and efficient. With 30 grayscale images per student, the model trained within 2–3 seconds for a batch of 10 students. This

quick turnaround makes the system ideal for institutions where new students are frequently added.

Recognition and attendance marking occurred in real time with a delay of less than 1 second from detection to confirmation. This low latency ensures a smooth experience for both students and faculty using the system.

4.3 Attendance Marking Accuracy

Attendance entries were recorded accurately in the CSV file, containing:

- Student ID
- Name
- Date and Time
- Entry Type (Entry, Exit, Absent)

The system ensured that no duplicate attendance entries were marked within a session. Additionally, the "Auto Mark Absentees" function correctly identified students who had not been marked present and recorded them as Absent. This feature was validated by manually cross-checking the list of registered users and the day's attendance log.

4.4 User Interface and Usability

The GUI built with Tkinter was tested for usability and responsiveness. All functionalities—registration, training, attendance logging, and data viewing—were easily accessible. The interface allowed smooth navigation, with clear prompts and validation messages for incorrect input. The theme switch between light and dark modes enhanced user comfort and accessibility in varying lighting conditions.

The teacher login portal added a layer of security for sensitive actions such as viewing full attendance logs, deleting student data, and marking absentees. This ensured that only authorized personnel could modify critical records.

4.5 Data Visualization and Management

Although the current system does not use complex dashboards, the CSV-based storage allowed users to open attendance records in Excel or other spreadsheet tools for

visualization. Each entry included precise timestamps, allowing for accurate tracking of attendance trends.

The system also supported viewing student databases through an integrated GUI window with sortable tables, enhancing administrative transparency and ease of record management.

4.6 Limitations and Challenges

While the system performed well under standard conditions, a few limitations were identified:

- **Lighting Sensitivity:** Recognition accuracy dropped under dim or overly bright conditions.
- **Pose Variation:** Side profiles or tilted head angles affected detection and recognition performance.
- **Hardware Dependency:** System performance may vary based on webcam quality and processing power.
- **Local Scope:** The current system operates offline and stores data locally, limiting scalability across multiple branches or institutions.

These limitations can be addressed in future enhancements by integrating deep learning-based models, multi-angle training data, and cloud storage solutions.

4.7 Summary

The experimental results validate the effectiveness of using facial recognition technology for attendance automation. The system delivered high accuracy, low latency, and a user-friendly experience, making it suitable for deployment in educational institutions. Despite a few constraints, the application successfully meets its core objectives and lays a strong foundation for further development.

CHAPTER 5 – SYSTEM DESIGN & ENHANCEMENTS

To deliver a robust and scalable solution, the Facial Recognition Attendance System has been designed with modular components, allowing each function to operate independently while contributing to a unified workflow. This chapter provides an overview of the system architecture, highlights the technologies used, explains testing strategies, discusses current limitations, and proposes enhancements for future development.

5.1 System Architecture

The system is logically divided into the following key modules:

1. User Registration Interface – Captures user data such as ID, name, and contact information.
2. Face Data Collection – Uses webcam and Haar Cascade to capture 30 face images per student.
3. Model Training Module – Trains the facial recognition model using the LBPH algorithm and stores it in the local system.
4. Recognition & Attendance Logging – Matches detected faces with stored profiles and logs the entry/exit in a CSV file.
5. Teacher Portal – Allows staff to mark absentees, view attendance logs, and manage the student database.
6. Graphical User Interface (GUI) – Provides a user-friendly frontend developed with Tkinter.

Each of these components is interconnected but independently functional, allowing easy updates, debugging, and expansion.

5.2 Technologies Used

- Programming Language: Python 3.x – Chosen for its simplicity and strong support for image processing, GUI development, and file handling.

- OpenCV – Used for real-time face detection and recognition (Haar Cascade + LBPH).
- Tkinter – Built-in Python library for creating a responsive and accessible GUI.
- Pandas – For structured data management, CSV operations, and attendance logs.
- NumPy – For efficient numerical processing and handling image data arrays.

These tools collectively provide a low-cost, open-source solution suitable for academic environments.

5.3 Testing and Evaluation

Each component of the system was tested individually as well as in full-stack configuration. The recognition accuracy, interface responsiveness, and data storage consistency were validated across multiple student profiles. The following test cases were conducted:

- Face recognition with new and trained students.
- Handling of duplicate or missing records.
- Model training speed and success validation.
- Attendance record correctness and formatting.
- Teacher dashboard operations including deletion and absentee marking.

All features operated successfully in controlled tests, and the application maintained consistent performance during multiple runs.

5.4 Limitations

While the system meets core requirements effectively, the following limitations were observed:

- The LBPH model, while lightweight, may not perform well in poor lighting or with significantly altered facial features.
- The solution does not yet include automated alert systems (e.g., sending emails/SMS to parents).

- The entire system is designed to work locally; it lacks real-time cloud-based access or backup.
- Only frontal face detection is supported, limiting accuracy for angled or partially obstructed faces.

5.5 Proposed Enhancements

To improve reliability, usability, and scalability, several enhancements are proposed:

- **Cloud Integration:** Store attendance records in real-time cloud databases for multi-institution access.
- **Email Alerts:** Notify parents/administrators automatically for absenteeism or late entry.
- **Multi-Camera Support:** Extend to cover different classrooms or entry points simultaneously.
- **Mobile App Extension:** Provide mobile access for administrators and teachers.
- **Deep Learning Integration:** Use CNN-based models (e.g., FaceNet, Dlib) to enhance recognition under complex conditions.

These future developments will significantly increase the utility and adoption of the system across broader contexts.

CHAPTER 6 – TECHNOLOGIES USED

The successful implementation of the Facial Recognition Attendance System relied on a combination of powerful and open-source technologies for computer vision, data processing, and graphical user interface (GUI) development. This chapter outlines the key technologies employed and the rationale behind their selection.

6.1 Programming Language

Python 3.x was chosen as the primary programming language due to its:

- Simplicity and readability.
- Extensive support for image processing and machine learning libraries.
- Strong community and library ecosystem.

Python enables rapid development, integration, and debugging, making it ideal for academic and prototype applications.

6.2 Libraries and Frameworks

The project utilizes the following libraries and frameworks:

- OpenCV:
 - Used for face detection and recognition.
 - Provides pre-trained models such as Haar Cascade Classifiers and the Local Binary Pattern Histogram (LBPH) face recognizer.
 - Enables webcam integration, image preprocessing, and real-time video analysis.
- Tkinter:
 - Python's built-in GUI toolkit.
 - Used to design and implement the entire graphical interface, including forms, buttons, labels, and tables.

- Lightweight and suitable for desktop applications.
- Pandas:
 - Provides data structures and functions to manage tabular data.
 - Used for reading and writing `users.csv` and `attendance.csv`.
 - Enables filtering, updating, and querying attendance records efficiently.
- NumPy:
 - Supports numerical operations and array handling.
 - Used to convert face image data into numerical arrays for training and prediction.
- PIL (Pillow):
 - Python Imaging Library used for image handling within the GUI (e.g., displaying face captures).
- MessageBox (tkinter.messagebox):
 - Used for user feedback, notifications, alerts, and confirmation dialogs throughout the application.

6.3 Development Environment

Development and testing were performed in the following environment:

- Operating System: Windows 10
- IDE: Visual Studio Code
- Dependencies Manager: pip (Python package installer)
- Camera: Built-in laptop webcam
- Execution: Script run directly via terminal or IDE

The project is lightweight and runs effectively on basic hardware configurations, ensuring broad accessibility.

6.4 File System and Data Storage

All data is stored in structured local directories:

- `/dataset/`: Contains captured grayscale face images during registration.
- `/trainer/`: Stores the trained LBPH model as `trainer.yml`.
- `users.csv`: Stores registered student data including ID, name, class, contact info.
- `attendance.csv`: Records attendance entries with timestamps and entry/exit status.

Data is stored in plain-text format (CSV) for easy compatibility with other tools (e.g., Excel, Google Sheets).

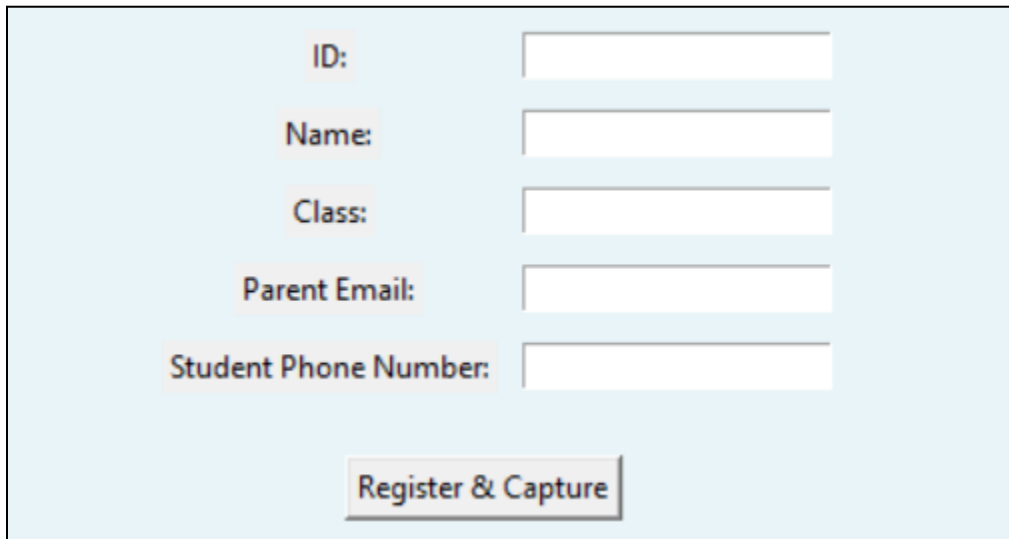
In conclusion, the technologies chosen for this project balance performance, simplicity, and usability. The use of Python and open-source libraries ensures flexibility and allows for future expansion into more sophisticated domains such as deep learning, cloud computing, or mobile app integration.

CHAPTER 7 – IMPLEMENTATION SNAPSHOTS

This chapter presents key visual snapshots from the implementation of the Facial Recognition Attendance System, highlighting each functional phase of the application—from data registration and model training to real-time attendance recognition and administrative features. These screenshots validate the working components of the system and illustrate its user-friendly interface and efficient functionality.

7.1 Registration and Face Capture

Snapshot: Student Registration Interface

A screenshot of a web-based registration form for a student. The form is set against a light blue background. It contains five text input fields, each preceded by a label: 'ID:', 'Name:', 'Class:', 'Parent Email:', and 'Student Phone Number:'. The labels are in a dark blue font, and the input fields are white with thin grey borders. Below these fields is a single button with the text 'Register & Capture' in a dark blue font, enclosed in a grey border.

Description:

The user enters details such as ID, Name, Class, Parent Email, and Phone Number in the registration form. Upon submission, the webcam is activated, and the system begins capturing facial data. A live feed is displayed along with a counter showing the number of captured images (out of 30). Once complete, the images are saved in the `dataset/` folder.

7.2 Model Training

Snapshot: Training Model Output (train.yml file)

```
%YAML:1.0
---
opencv_lbphfaces:
  threshold: 1.7976931348623157e+308
  radius: 1
  neighbors: 8
  grid_x: 8
  grid_y: 8
  histograms:
    - !!opencv-matrix
      rows: 1
      cols: 16384
      dt: f
      data: [ 4.30839024e-02, 1.81405898e-02, 0., 2.26757373e-03,
        2.04081628e-02, 2.26757373e-03, 0., 1.13378689e-02, 0., 0.,
        0., 0., 2.26757373e-03, 4.53514745e-03, 6.80272095e-03,
        4.53514745e-03, 2.04081628e-02, 6.80272095e-03, 0., 0.,
        4.53514745e-03, 0., 0., 4.53514745e-03, 6.80272095e-03, 0.,
        0., 0., 2.04081628e-02, 4.53514745e-03, 1.13378689e-02,
        1.36054419e-02, 2.26757373e-03, 0., 0., 0., 0., 0., 0., 0., 0.,
```

Description:

After capturing facial images for all students, the "Train Model" button initiates the training process.

7.3 Real-Time Face Recognition and Attendance Marking

Snapshot: Student Entry/Exit Recognition

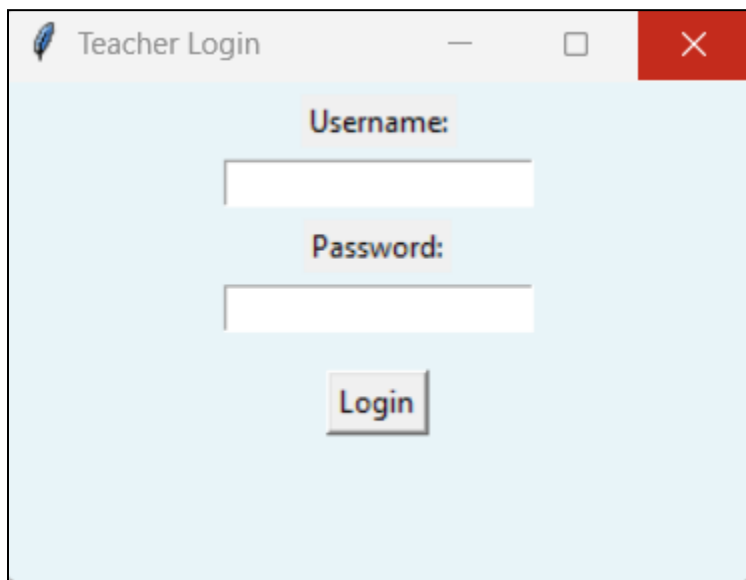


Description:

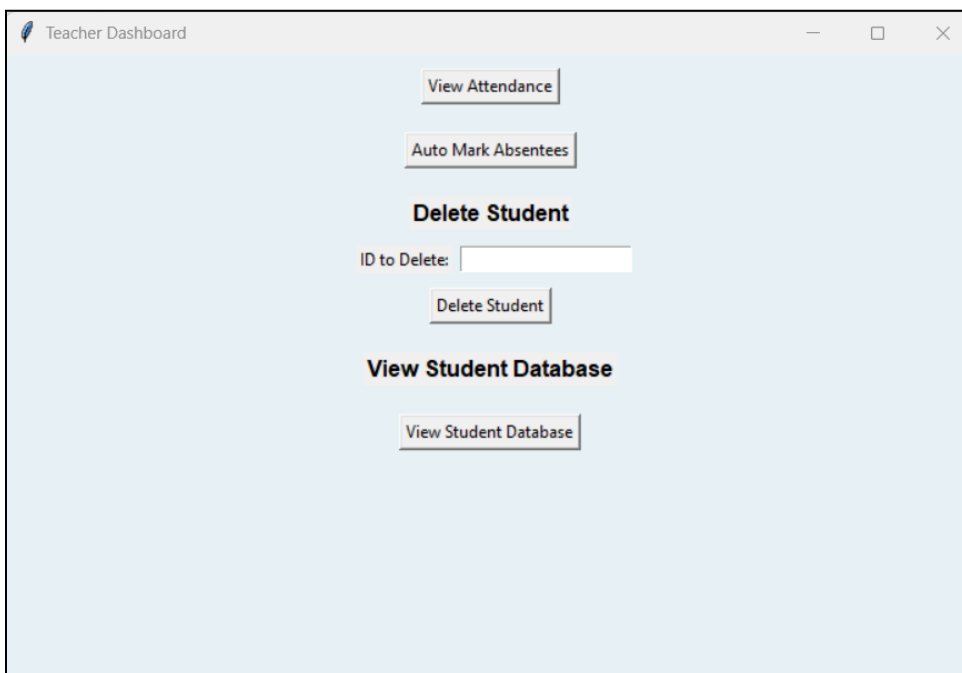
The system activates the webcam to identify a face in real time. When a registered student is recognized, their name and ID are matched, and their attendance is marked with the corresponding entry type ("Entry" or "Exit") along with the date and time. A message box confirms successful attendance logging.

7.4 Teacher Login and Dashboard

Snapshot: Secure Teacher Portal



A screenshot of a web browser window titled "Teacher Login". The window has a light blue background. It contains two text input fields: the first is labeled "Username:" and the second is labeled "Password:". Below these fields is a button labeled "Login". The window has standard OS controls (minimize, maximize, close) in the top right corner.



A screenshot of a web browser window titled "Teacher Dashboard". The window has a light blue background. It contains several buttons: "View Attendance", "Auto Mark Absentees", "Delete Student", "View Student Database", and "Delete Student". Below the "Delete Student" button is a text input field labeled "ID to Delete:". Below the "View Student Database" button is another text input field labeled "View Student Database". The window has standard OS controls (minimize, maximize, close) in the top right corner.

Description:

The teacher accesses advanced features via a login window with predefined credentials. Upon successful authentication, the dashboard allows the teacher to:

- View full attendance records.
- Automatically mark absentees.
- Delete student records based on ID.
- View the complete student database in a sortable table format.

7.5 View Attendance Records

Snapshot: Attendance Table Viewer

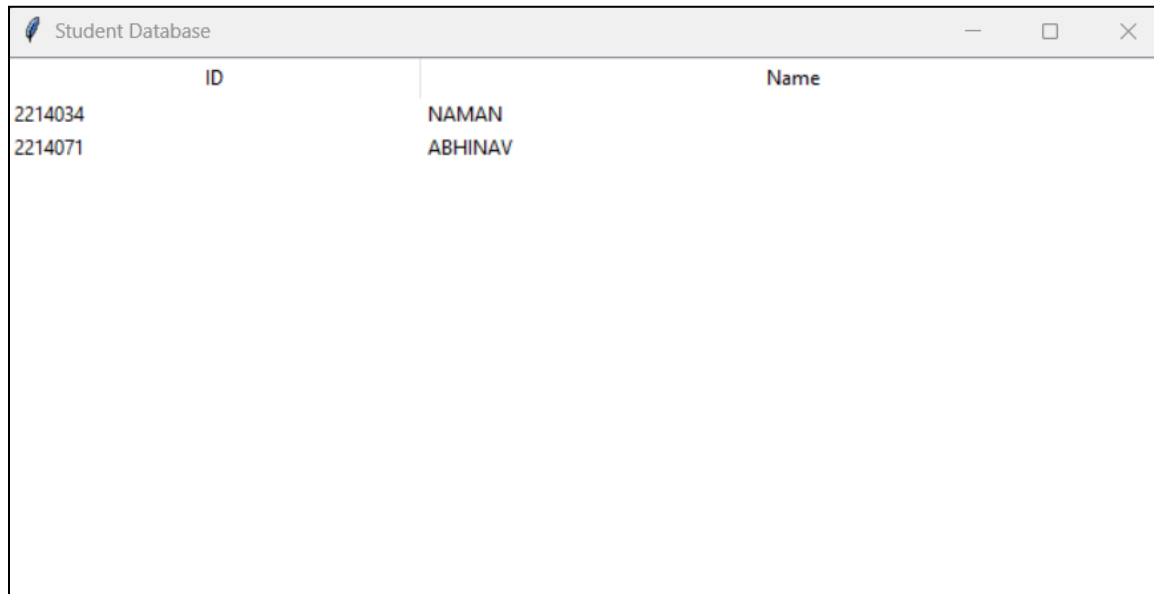
ID	Name	DateTime	Type
221	abh	2025-05-01 07:18:20	Entry
221	abh	2025-05-01 07:18:26	Exit
33	ab	2025-05-01 07:40:26	Entry
33	ab	2025-05-01 08:11:51	Entry
23	abh	2025-05-01 08:12:24	Entry
23	abh	2025-05-01 08:12:39	Entry
2214034	NAMAN	2025-05-01 08:19:53	Entry
2214034	NAMAN	2025-05-01 08:19:58	Exit
2214071	ABHINAV	2025-05-01 08:20:28	Entry
2214034	NAMAN	2025-05-01 08:20:34	Exit

Description:

This window displays all logged attendance records including student ID, name, timestamp, and attendance type. It enables the teacher to monitor student attendance over time and ensures transparency in attendance logging.

7.6 Student Database Viewer

Snapshot: Student Data Table



ID	Name
2214034	NAMAN
2214071	ABHINAV

Description:

This module provides a list of all registered students along with their IDs and names. It offers a quick overview and ensures data integrity within the system.

REFERENCES

1. <https://docs.opencv.org/>
2. Ahonen, T., Hadid, A., & Pietikainen, M. (2006). *Face Description with Local Binary Patterns: Application to Face Recognition*. IEEE Transactions on Pattern Analysis and Machine Intelligence.
3. Viola, P., & Jones, M. (2001). *Rapid Object Detection using a Boosted Cascade of Simple Features*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
4. <https://towardsdatascience.com/face-recognition-how-lbph-works-90ec258c3d6b>
5. <https://pillow.readthedocs.io/en/stable/>